

UNIT 3 ASSIGNMENT REPORT

Unit 3: Sorting Algorithms – Report

Overview

This unit focuses on understanding, implementing, and analyzing various sorting algorithms. It covers both basic and advanced techniques, comparing their efficiency, stability, and practical applications.

4.1 Experiment 14: $O(n^2)$ Sorts + Counting Sort

Objective

To study simple quadratic sorting algorithms and introduce a non-comparison-based sorting method.

Algorithms Covered

- Bubble Sort
- Selection Sort
- Counting Sort

Key Points

- Bubble and Selection Sort have $O(n^2)$ time complexity.
 - Counting Sort works in $O(n + k)$ time.
 - Counting Sort is **not comparison-based** and is efficient for small ranges.
-

4.2 Experiment 15: Insertion Sort

Objective

To understand the working and advantages of Insertion Sort.

Key Features

- Time Complexity:

- Best Case: **$O(n)$** (nearly sorted array)
 - Worst Case: **$O(n^2)$**
- **Stable sorting algorithm**
- Efficient for **small or nearly sorted datasets**

Conclusion

Insertion Sort performs very well when the input is already almost sorted.

4.3 Experiment 16: Merge Sort

Objective

To implement divide-and-conquer sorting.

Key Features

- Time Complexity: **$O(n \log n)$**
- **Stable sorting algorithm**
- Uses **extra space ($O(n)$)**

Working

- Divide array into halves
 - Sort each half recursively
 - Merge sorted halves
-

4.4 Experiment 17: Quick Sort

Objective

To study partition-based sorting.

Key Features

- Average Time Complexity: **$O(n \log n)$**
- Worst Case: **$O(n^2)$**
- **Not stable**
- In-place sorting (no extra space needed)

Concept

- Choose a pivot
- Partition elements

- Recursively sort partitions
-

4.5 Experiment 18: Heap Sort

Objective

To understand heap-based sorting.

Key Features

- Time Complexity: $O(n \log n)$
- **Not stable**
- Uses **Heap Data Structure**

Working

- Build max heap
 - Extract elements one by one
 - Place them in sorted order
-

4.6 Experiment 19: Benchmark Harness

Objective

To compare performance of sorting algorithms.

Process

- Use different datasets:
 - Random
 - Sorted
 - Reverse sorted
- Measure execution time
- Create comparison table

Observation

- $O(n \log n)$ algorithms perform better for large inputs
 - Simple sorts are useful for small datasets
-

4.7 Unit 3 Assignment (Student Work Summary)

Tasks Completed

- Implemented all sorting algorithms
- Compared time and space complexity
- Analyzed stability and efficiency
- Tested algorithms on multiple datasets

Learning Outcomes

- Understanding of different sorting techniques
- Ability to choose appropriate algorithm
- Knowledge of time-space trade-offs

Conclusion

This unit provides a complete understanding of sorting algorithms ranging from simple to advanced. It highlights:

- Performance differences
- Importance of stability
- Practical use cases

Sorting is a fundamental concept that plays a crucial role in data processing and algorithm design.
